EigenLayer

EigenDA x Arbitrum Integration

Security Assessment Report

Version: 2.0

November, 2024

Contents

Introduction 2
- Disclaimer . . . 2
- Document Structure . . . 2
- Overview . . . 2

Security Assessment Summary 3
- Scope . . . 3
- Approach . . . 3
- Coverage Limitations . . . 4
- Findings Summary . . . 4

Detailed Findings 5

Summary of Findings 6
- Blob Length Check Bypass . . . 7
- Arbitrary KZG Proof Construction In One Step Prover . . . 9
- Unused Hash Field In KZG Proof Verification . . . 11
- Reading Sequencer Messages From Calldata Breaks With `EIP-7702` . . . 12
- Missing `tx.origin` Check When Adding Batches Through EigenDA . . . 14
- Insufficient Length Check When Encoding Blobs . . . 15
- Array Slicing Without Length Validation . . . 16
- Lack Of Validation In `RecoverPayloadFromBatch()` . . . 17
- Miscellaneous General Comments . . . 18

A Vulnerability Severity Classification 19

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the EigenLayer components in scope. The review focused solely on the security aspects of the Solidity implementation of the contract, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the smart contract. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the EigenLayer components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see Vulnerability Severity Classification), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the EigenLayer components in scope.

Overview

EigenDA is a data availability protocol that can be used to temporarily store data. This review focuses on a fork of the layer 2 optimistic rollup, Abitrum Nitro. The fork adds the functionality to use EigenDA as a data availability layer.

Updates have been made to the sequencer to allow posting of EigenDA batches. The batches are validated onchain through an EigenDA certificate. Offchain validators can fetch the required batch data through a query to an EigenDA proxy using the sequenced certificates.

Additionally, changes have been made to the fraud proof, to handle proving of EigenDA batches. The required changes include modifications to the preimage opcode to include KZG point verification.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the Layr Labs forks of nitro-contracts and nitro repositories..

The scope of this time-boxed review was strictly limited to the files at nitro-contracts@3318395 and nitro@b50fff8.

The fixes of the identified issues were assessed at nitro-contracts@03ef06f and nitro@49a3969.

Note: the scope was limited to changes made between the origin Arbitrum repositories and the Layr Labs updates.

Approach

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

For a more detailed, but non-exhaustive list of examined vectors, see [1, 2].

To support this review, the testing team also utilised the following automated testing tools:

- Mythril: `https://github.com/ConsenSys/mythril`
- Slither: `https://github.com/trailofbits/slither`
- Surya: `https://github.com/ConsenSys/surya`
- Aderyn: `https://github.com/Cyfrin/aderyn`

For the Golang libraries and modules, the review focused on internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Go runtime. The manual review explored known Golang antipatterns such as integer overflow, floating point underflow, deadlocking, race conditions, memory and CPU exhaustion attacks and a multitude of panics including but not limited to `nil` pointer deferences, index out of bounds and calls to `panic()`.

To support this review, the testing team also utilised the following automated testing tools:

- golangci-lint: `https://golangci-lint.run/`
- vet: `https://pkg.go.dev/cmd/vet`
- errcheck: `https://github.com/kisielk/errcheck`

For the Rust components, the review focused on the intended functionality and correct implementation with respect to the underlying functionality of the Rust language. The manual review explored known Rust antipatterns

such as handling of `unsafe` code blocks, integer overflow, floating point underflow, deadlocking, error handling memory and CPU exhaustion attacks and a multitude of panics including but not limited to index out of bounds, calls to `panic!()`, `unwrap()` and `unreachable!()`.

To support this review, the testing team also utilised the following testing tools:

- Clippy linting: https://doc.rust-lang.org/stable/clippy/index.html
- Cargo Audit: https://github.com/RustSec/rustsec/tree/main/cargo-audit
- Cargo Outdated: https://github.com/kbknapp/cargo-outdated
- Cargo Geiger: https://github.com/rust-secure-code/cargo-geiger
- Cargo Tarpaulin: https://crates.io/crates/cargo-tarpaulin

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 9 issues during this assessment. Categorised by their severity:

- Critical: 2 issues.
- Medium: 2 issues.
- Low: 3 issues.
- Informational: 2 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the EigenLayer components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each issue by the matrix given in the Appendix: Vulnerability Severity Classification.

A number of additional properties of the contracts, including gas optimisations, are also described in this section and are labelled as "informational".

Each vulnerability is also assigned a **status**:

- ***Open:*** the issue has not been addressed by the project team.
- ***Resolved:*** the issue was acknowledged by the project team and updates to the affected contract(s) have been made to mitigate the related risk.
- ***Closed:*** the issue was acknowledged by the project team but no further actions have been taken.

Summary of Findings

ID	Description	Severity	Status
EDAA-01	Blob Length Check Bypass	Critical	Resolved
EDAA-02	Arbitrary KZG Proof Construction In One Step Prover	Critical	Resolved
EDAA-03	Unused Hash Field In KZG Proof Verification	Low	Resolved
EDAA-04	Reading Sequencer Messages From Calldata Breaks With `EIP-7702`	Medium	Closed
EDAA-05	Missing `tx.origin` Check When Adding Batches Through EigenDA	Medium	Resolved
EDAA-06	Insufficient Length Check When Encoding Blobs	Low	Resolved
EDAA-07	Array Slicing Without Length Validation	Low	Resolved
EDAA-08	Lack Of Validation In `RecoverPayloadFromBatch()`	Informational	Closed
EDAA-09	Miscellaneous General Comments	Informational	Closed

EDAA-01	Blob Length Check Bypass		
Asset	`OneStepProverHostIo.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

An unsafe casting issue in `OneStepProverHostIo` allows a prover to manipulate the `preimageLength` to modify `z`.

The preimage length check is performed on a variable cast down to 32 bits (`uint32`) from 256 bits. However, when the length is used in `preimageLength`, the entire 256 bits are used. Therefore, an attacker can make use of the leading 224 bits which are not validated.

The `preimageLength` is used to calculate the value of `z`.

```
    uint256 length = uint32(uint256(bytes32(kzgProof[352:384]))); // @audit casting from 256bits to 32bits (then back to 256bits)
    uint32 length_u32 = uint32(length); // @audit casting to 32bits again
    ...
    require(
        keccak256(abi.encodePacked(kzgProof[224:288], length_u32)) == leafContents,
        "BN254_KZG_PROOF_WRONG_HASH"
    );
...
// read the preimage length
uint256 preimageLength = uint256(bytes32(kzgProof[352:384])); // @audit uses the whole 256bits


// If preimageOffset is greater than or equal to the blob size, leave extracted empty and call it here.
if (preimageOffset < preimageLength) {
    // preimageOffset was required to be 32 byte aligned above
    uint256 tmp = preimageOffset / 32;
    // First, we get the root of unity of order 2**fieldElementsPerBlob.
    // We start with a root of unity of order 2**32 and then raise it to
    // the power of (2**32)/fieldElementsPerBlob to get root of unity we need.
    uint256 rootOfUnityPower = ((1 << 28) / preimageLength) * 32; // @audit can round to zero if `preimageLength > 2^28`
    // Then, we raise the root of unity to the power of bitReversedIndex,
    // to retrieve this word of the KZG commitment.
    rootOfUnityPower *= tmp;
    // z is the point the polynomial is evaluated at to retrieve this word of data
    uint256 z = modExp256(
        BN_254_PRIMITIVE_ROOT_OF_UNITY,
        rootOfUnityPower, // @audit when this is zero, modExp256() returns one
        BN254.FR_MODULUS
    );
    require(bytes32(kzgProof[32:64]) == bytes32(z), "KZG_PROOF_WRONG_Z"); // @audit can set z = 1 with the malicious length
```

An example of an attack consists in taking a genuine length, say 100, and adding a large value say 2^{255}.

Therefore, `length_u32 = 100` and `preimageLength = 2**255 + 100`.

This issue is present in the line `rootOfUnityPower = ((1 << 28) / preimageLength) * 32`; that is because `rootOfUnityPower` would round to zero.

Hence, the `modExp256()` would have an exponent zero, returning `z = 1`.

Recommendations

A mitigation to the issue is to first decode the `length` as a `uint256` then verify that it is less than 2^{28}.

Resolution

The EigenDA team has implemented a fix for this issue by casting the `preimageLength` to `uint32` before using it in the `rootOfUnityPower` calculation. Therefore, the excess bytes of the proof are discarded in both cases.

This issue has been resolved in PR #31.

EDAA-02	Arbitrary KZG Proof Construction In One Step Prover		
Asset	`OneStepProverHostIo.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Critical	Impact: High	Likelihood: High

Description

The function `executeReadPreImage()` allows arbitrary KZG proofs to be constructed for EigenDA preimages.

The proof verification is the pairing `e((P - y) + gamma . (alpha - z), G2) = e((Q + gamma), (alpha - z))`, where the last term `(alpha - z)` for the G2 curve is passed as untrusted input from the user.

There lacks significant validation of `(alpha - z)` on G2 to ensure it indeed represents the shared secret subtracted by `z`.

The pairing implements `gamma` as a blinding factor which is intended as an optimisation to validate `(alpha -z)` on G2 in the same pairing as the KZG verification. This blinding factor allows combining the following two pairings:

- `e((P - y), G2) = e(Q, (alpha - z))`
- `e(gamma . (alpha - z), G2) = e(gamma, (alpha - z))` where `alpha - z` on G1 is calculated from each value.

The limitation of the current implementation is that the blinding factor `gamma` is not bound to the proof `Q`. The value of `gamma` is the hash of `z, y, P, (alpha - z) on G2`, noting that it does not include `Q`. Therefore, an attacker is able to craft the proof `Q` after calculating a value for `gamma`.

The following describes the relevant exploitation steps:

1. Commit to any point P
2. Set `(alpha - z)` on G2 to the value of the generator for that group.
3. Pick any arbitrary value of `y`
4. Now `y`, `(alpha - z) on G2` and `P` and `z` are known, thus `gamma` can be calculated.
5. To make a malicious proof, set `Q = -gamma + (P - y) + gamma . (alpha - z)`. These operations are all done on the G1 curve, thus the discrete log of `alpha` does not need to be known.

The construction of the proof above will result in the following pairings.

- `LHS = e(P - y + gamma.(alpha - z), G2)`
- `RHS = e(P - y + gamma.(alpha - z), G2)`

Hence, it is possible to construct a proof for any arbitrary value of `y` for a commitment to `P` at `z`.

Recommendations

To resolve the issue, the blinding factor `gamma` should include the proof `Q` in the hash. Including `Q` in the calculation for `gamma` will prevent it being calculated before the proof.

Then the extended pairing function will ensure that `(alpha - z)` on G1 matches `(alpha - z)` on G2.

Resolution

The EigenDA team has implemented a fix for this issue by including `Q` and `(alpha - z)` on G1 in the hash used to calculate `gamma`.

This issue has been resolved in PR #29.

EDAA-03	Unused Hash Field In KZG Proof Verification		
Asset	`OneStepProverHostIo.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

In `executeReadPreImage()`, there is no validation of the first 32 bytes of `kzgProof`. These bytes are set to be the commit hash but never checked.

The commit hash is validated through `keccak256(abi.encodePacked(kzgProof[224:288], length_u32)) == leafContents`, thus the impact is rated as low.

Recommendations

The first 32 bytes of `kzgProof` are not required on-chain, consider removing these bytes from the proof.

Resolution

The first 32 bytes of `kzgProof` has been removed from the proof in `OneStepProverHostIo.sol` and during proof generation in `nitro/arbitrator/prover/src/kzgbn254.rs`.

The EigenDA team has resolved this issue in PRs #31 and #45.

EDAA-04	Reading Sequencer Messages From Calldata Breaks With `EIP-7702`		
Asset	`nitro/arbnode/sequencer_inbox.go, nitro-contracts/src/bridge/SequencerInbox.sol`		
Status	**Closed:** See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

Malicious sequencers can use `EIP-7702` to manipulate calldata that can be used to send arbitrary sequencer batches to validators.

Sequencer L2 batches where the data location is in the transaction calldata or EigenDA are handled in `sequencer_inbox.go::getSequencerData()`. The batch data or EigenDA certificate is fetched directly from the transaction calldata. This relies on a security assumption that the sequencer is an EOA that directly calls the `SequencerInbox` contract.

`EIP-7702` introduced in the upcoming Ethereum Prague upgrade allows EOAs to set account code. This means that a malicious sequencer can now spoof their calldata while submitting batches to the sequencer inbox, such that Arbitrum validators will fetch the wrong batch data or EigenDA certificate from the calldata.

Note that this attack can also be performed with the `addSequencerL2BatchFromOrigin()` function even though the function ensures that `msg.sender == tx.origin`, as `EIP-7702` allows EOAs to behave like smart contracts.

A malicious sequencer can perform the attack as follows:

1. Sign and post an `EIP-7702` transaction to set their code to a malicious contract that has the same function signatures as the `SequencerInbox` contract. The malicious contract should also have a function that allows the malicious sequencer to store EigenDA certificates in storage.
2. Call `addSequencerL2BatchFromEigenDA()` on the malicious sequencer address with an empty/malicious EigenDA certificate. This function in the malicious contract should ignore the `cert` parameter in the calldata, and instead fetch the certificate from storage to call the `SequencerInbox` contract.
3. Validators will fetch the empty/malicious EigenDA certificate from the transaction calldata instead of the actual certificate posted to `SequencerInbox`. The validator will be unable to process and validate the batch.

Depending on what the malicious sequencer decides to post as calldata for the certificate, validators can process invalid batches, or pause the L2 chain.

Recommendations

To prevent this attack, both `addSequencerL2BatchFromOrigin()` and `addSequencerL2BatchFromEigenDA()` should ensure that the following conditions are met:

1. `msg.sender == tx.origin`
2. `address(msg.sender).code.length == 0`

Resolution

The EigenDA team has implemented the recommended changes above to `addSequencerL2BatchFromEigenDA()` in PR #28.

Fixes to `addSequencerL2BatchFromOrigin()` were not implemented with the intention of adding them in the future once they are present in Arbitrum's `nitro-contracts` repo.

EDAA-05	Missing `tx.origin` Check When Adding Batches Through EigenDA		
Asset	`nitro-contracts/src/bridge/SequencerInbox.sol`		
Status	**Resolved:** See Resolution		
Rating	Severity: Medium	Impact: High	Likelihood: Low

Description

The `addSequencerL2BatchFromEigenDA()` function in `SequencerInbox.sol` does not check `msg.sender == tx.origin`. Hence, it is possible for a malicious sequencer to be a contract, which may allow the transaction originator to spoof the EigenDA certificate that are fetched by validators.

The `msg.sender == tx.origin` ensures that the sequencer is not a smart contract. This is needed as validators read the EigenDA certificate from the transaction input data, as opposed to the calldata of the external call made to the `SequencerInbox` contract.

In the case where the sequencer is a smart contract that is called by a malicious operator, the operator can perform a similar attack to the one described in EDAA-04, such that validators will fetch an incorrect or invalid EigenDA certificate for that batch.

Depending on the malicious certificate payload, this may lead to the L2 chain being halted as the batch cannot be processed and validated, or the validator incorrectly posting or challenging the real batch.

This issue has a low likelihood of being exploited, as it requires the rollup owner to add a sequencer that is a malicious contract.

Recommendations

A check should be performed to ensure the sender is an EOA. This can be achieved through the recommendation in EDAA-04.

Resolution

The EigenDA team has added a check for `msg.sender == tx.origin` in `addSequencerL2BatchFromEigenDA()`.

This issue has been fixed in PR #28.

EDAA-06	Insufficient Length Check When Encoding Blobs		
Asset	`eigenda/serialize.go`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The function `encodeBlob()` does not perform validation of the incoming blob length before encoding it.

```go
func encodeBlob(rawData []byte) ([]byte, error) {
    codecBlobHeader := make([]byte, 32)
    // first byte is always 0 to ensure the codecBlobHeader is a valid bn254 element
    // encode version byte
    codecBlobHeader[1] = byte(0x0)

    // encode length as uint32
    binary.BigEndian.PutUint32(codecBlobHeader[2:6], uint32(len(rawData))) // uint32 should be more than enough to store the length
        ↪ (approx 4gb)
```

The length of `rawData` is cast to a `uint32` and encoded, without validating the length of the data fits in a `uint32`.

A likelihood of low has been assigned, as this function is called on data that is fetched from the EigenDA proxy, where there is a 2MB limit on EigenDA blob sizes Thus, it is unlikely to be malformed data.

Recommendations

It is recommended to validate the length is less than a `uint32` before casting.

Resolution

The EigenDA team has implemented the recommended fix in PR #39.

EDAA-07	Array Slicing Without Length Validation		
Asset	`arbnode/sequencer_inbox.go`		
Status	**Resolved:** See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

A node validating the sequencer inbox checks the `calldata` of a transaction is indexed at position 4 without first validating the length.

The following code snippet fetches the `calldata` from a transaction and takes a slice starting at index 4. If the calldata has length less than 4 this will result in an index out of bounds panic.

```
case batchDataEigenDA:
    tx, err := arbutil.GetLogTransaction(ctx, client, m.rawLog)
    if err != nil {
        return nil, err
    }

    calldata := tx.Data()

    args := make(map[string]interface{})
    err = addSequencerBatchFromEigenDACallABI.Inputs.UnpackIntoMap(args, calldata[4:]) // @audit should check calldata length is
        ↪ greater than 4
    if err != nil {
        return nil, err
    }

    certBytes, err := interfaceToBytesJSON(args["cert"])
    if err != nil {
        return nil, err
    }
```

It is noted that Solidity uses the first 4 bytes of a transaction as a function selector. Since the onchain `SequencerInbox` contract does not have a fallback function, then all transactions which do no revert will have more than 4 bytes of `calldata`.

However, if cases arise such as EDAA-04 where a malicious sequencer can manipulate the `calldata`, this would allow the sequencer to trigger panics on validator nodes.

Recommendations

Perform length checks before slicing arrays.

Resolution

The EigenDA team has added the calldata length check in PR #39.

EDAA-08	Lack Of Validation In `RecoverPayloadFromBatch()`
Asset	`eigenda/reader.go`
Status	**Closed:** See Resolution
Rating	Informational

Description

There is insufficient validation of the `sequencerMsg` data that is passed to the EigenDA reader in `RecoverPayloadFromBatch()`.

```
func (d *readerForEigenDA) RecoverPayloadFromBatch(
    ctx context.Context,
    batchNum uint64,
    batchBlockHash common.Hash,
    sequencerMsg []byte,
    preimageRecorder daprovider.PreimageRecorder,
    validateSeqMsg bool,
) ([]byte, error) {
    return RecoverPayloadFromEigenDABatch(ctx, sequencerMsg[sequencerMsgOffset:], d.readerEigenDA, preimageRecorder, "binary")
}
```

The function `RecoverPayloadFromBatch()` does not perform validation to ensure `sequencerMsg` has a length of `sequencerMsgOffset` before slicing the array.

Additionally, there is no validation to ensure this is an EigenDA batch message.

The issue is raised as informational as the `sequencer_inbox.Serialize()` calculations append 40 bytes of header data and `sequencer_inbox::getSequencerData()` appends a header flag.

Recommendations

Two checks should be added to improve robustness of the protocol in `RecoverPayloadFromBatch()`:

1. ensure `sequencerMsg` is longer than `sequencerMsgOffset`.
2. validate the 40th byte is a valid header byte via the function `IsValidHeaderByte()`.

Resolution

The EigenDA team has chosen to not implement fixes for this issue to maintain consistency with the rest of the DAP readers.

Additionally, validation for `sequencerMsg` length and batch message type is performed in `arbstate/inbox.go::parseSequencerMessage()`.

EDAA-09	Miscellaneous General Comments
Asset	All contracts
Status	**Closed:** See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. **Typos**
   Related Asset(s): eigenda/serialize.go
   The function `encodeBlob()` has a spelling mistake on line 75 in `reassgin` which should say `reassign`.

   ```
   // append raw data; reassgin avoids copying
   ```

   Update the comments as needed.
2. **Use Constants Rather Than Hard Coded Values**
   Related Asset(s): eigenda/serialize.go, eigenda/proxy.go
   The function `serialize.go::encodeBlob()` uses `byte(0x0)` to represent the EigenDA blob version.

   ```
   codecBlobHeader := make([]byte, 32)
    // first byte is always 0 to ensure the codecBlobHeader is a valid bn254 element
    // encode version byte
    codecBlobHeader[1] = byte(0x0) // @audit consider using a constant here for version
   ```

   The function `proxy.go::Get()` also uses a hard coded value of `{[}{]}byte\{0x0\}` as the version.

   ```
   commitWithVersion := append([]byte{0x0}, commitment...)
   ```

   It is recommended yo use a constant rather than hard coded value to improve maintainability and readability.
3. **Constant Without Explanation**
   Related Asset(s): eigenda/eigenda.go
   The constant variable `sequencerMsgOffset` is set on line [13] without comments as explanation for the value.
   The value is set to `41` which represents 40 bytes of header data and 1 byte as the header flag.
   It is recommended to add a comment to describe where these values come from.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The EigenDA team has fixed the typo in the first comment in PR #39.

The second and third comments have been acknowledged.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurance. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	Likelihood: Low	Likelihood: Medium	Likelihood: High
High	Medium	High	Critical
Medium	Low	Medium	High
Low	Low	Low	Medium

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

References

[1] Sigma Prime. Solidity Security. Blog, 2018, Available: https://blog.sigmaprime.io/solidity-security.html. [Accessed 2018].

[2] NCC Group. DASP - Top 10. Website, 2018, Available: http://www.dasp.co/. [Accessed 2018].

σ'